# 数値計算法 第2回

## 誤差

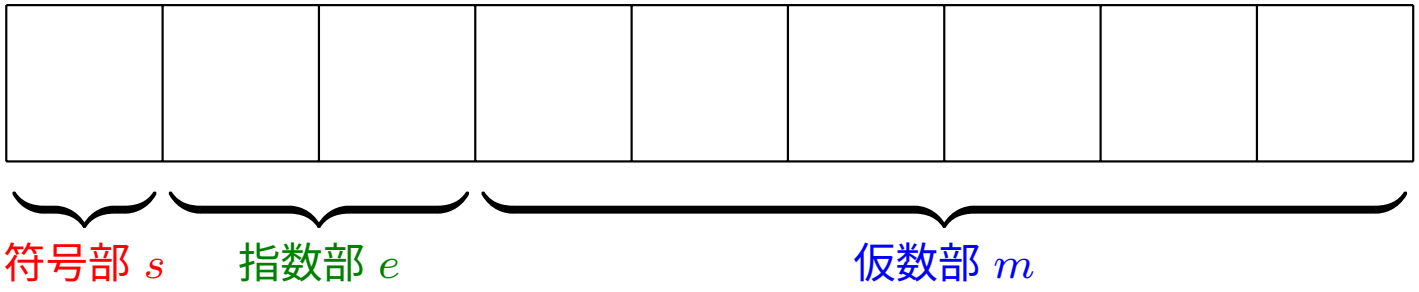
濱本 雄治 [1]

情報工学部 情報通信工学科

2026 年 4 月 20 日

[1] hamamoto@c.oka-pu.ac.jp

# 復習: 10 進 8 桁の仮想的な浮動小数点数

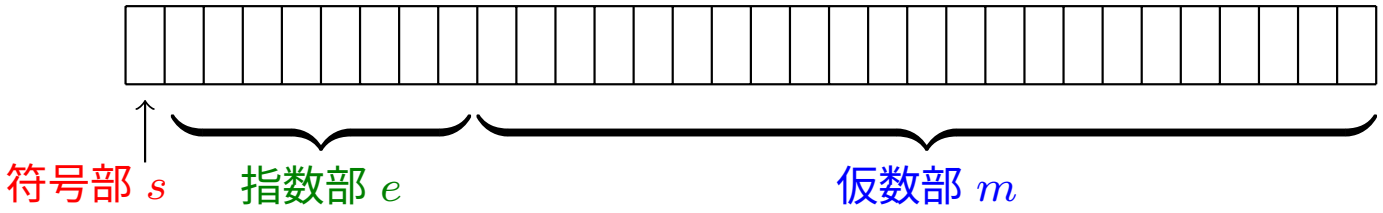



- 符号も含めた小数

$$(-1)^s \times 0.m \times 10^{e-50} \qquad \text{(正規化)}$$

- $s = 0, 1$ なので符号部は 1 ビット (2 進数 1 桁) でよい
- $m$ は 6 つの数字の並びを表し、一番左の数字は 0 でない

# 復習: IEEE 単精度 (32 ビット) 浮動小数点数




- 符号部は 1 ビットの 2 進数で $s = 0, 1$
- 指数部は 8 ビットの 2 進数で $(00000000)_2 \leq e \leq (11111111)_2$
- 仮数部は残りの 23 ビット分だけ $0$ or $1$ を並べて表現
- $0 < e < 255$ のとき

$$(-1)^s \times (1.m)_2 \times 2^{e-127} \qquad \text{(正規化)}$$

- $e = 0, 255$ のとき

| | $m = 0$ | $m \neq 0$ |
|---|---|---|
| $e = 0$ | 符号付きゼロ $(-1)^s \times 0$ | $(-1)^s \times (0.m)_2 \times 2^{-126}$ |
| $e = 255$ | 符号付き無限大 $(-1)^s \times \infty$ | 非数 (Not a number, NaN) |

# 前回の小テストの解説

10 進数の $-1.5$ を IEEE 形式単精度浮動小数点数のビット列で表せ。ただし 0 が $n$ 桁続くとき $\underbrace{0\cdots0}_{n\text{ 桁}}$ のように略記してよい。

$$-1.5 = (-1)^{1} \times (1.1)_2 \times 2^{127-127}$$

$$\therefore s = 1, \quad e = 127 = 2^7 - 1 = (0\underbrace{1\cdots1}_{7\text{ 桁}})_2, \quad m = 1\underbrace{0\cdots0}_{22\text{ 桁}}$$

| | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

符号部 $s$　指数部 $e$　仮数部 $m$

# 第2回の目標

# 誤差

- コンピュータの浮動小数点数は近似値
- 近似値と真の値の単純な差を絶対誤差と呼ぶ

$$\text{絶対誤差} = \text{近似値} - \text{真の値}$$

- 絶対誤差と真の値の比を相対誤差と呼ぶ

$$\text{相対誤差} = \frac{\text{(絶対) 誤差}}{\text{真の値}} = \frac{\text{近似値}}{\text{真の値}} - 1$$

  - 誤差の絶対値より相対値の方が重要 $\rightarrow$ 通常は相対誤差を用いる

# 相対誤差の例

- 円周率の真の値 $\pi = 3.141592\cdots$ を
  - $\pi \simeq 3.14$ と近似すると

$$\text{絶対誤差} = 3.14 - 3.141592\cdots = -0.001592\cdots$$

$$\text{相対誤差} = \frac{-0.001592}{3.141592\cdots} = -0.000506\cdots$$

  - $\pi \simeq 3$ と近似すると

$$\text{絶対誤差} = 3 \quad - 3.141592\cdots = -0.141592\cdots$$

$$\text{相対誤差} = \frac{-0.141592}{3.141592\cdots} = -0.0450\cdots$$

# 精度

- 近似値と真の値の一致の程度を表す指標を精度と呼ぶ

## 精度を見積もる方法 1

相対誤差の小数点以下何桁目にゼロでない数字が現れるかを調べる

$$\text{相対誤差} = -0.000506 \quad \rightarrow \quad \text{誤差は 0.1\%未満}$$

相対誤差 0.0001 と 0.0009 は同じ精度?

## 精度を見積もる方法 2

精度の桁数を次式で定義

$$-\log_{10} |\,\text{相対誤差}\,|$$

手計算には常用対数表が必要

# 精度の例

- 円周率の真の値 $\pi = 3.141592\cdots$ を
  - $\pi \simeq 3.14$ と近似すると

$$\text{相対誤差} = -0.000506\cdots \qquad (4\text{ 桁目にゼロでない数字})$$

$$\therefore \text{精度は } 4 \text{ 桁}$$

$$-\log_{10}|\text{相対誤差}| \simeq 3.29$$

  - $\pi \simeq 3$ と近似すると

$$\text{相対誤差} = -0.0450\cdots \qquad (2\text{ 桁目にゼロでない数字})$$

$$\therefore \text{精度は } 2 \text{ 桁}$$

$$-\log_{10}|\text{相対誤差}| \simeq 1.34$$

# 誤差の種類

| 誤差 | 原因 |
|---|---|
| 丸め誤差 | 有効桁数が有限 |
| 情報落ち | 値に大きな差がある 2 数の加減算 |
| 桁落ち | 値が近い 2 数の減算 |
| 打切り誤差 | 無限回の演算を有限回で打ち切り |
| 離散化誤差 | 連続量を離散値で近似 |

# 丸め誤差

- コンピュータの有効桁数が有限であることに起因
- 1234.5678 を 10 進 8 桁の浮動小数点数で表すと

$$1234.5678 = 0.12345678 \times 10^{54-50}$$

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 5 | 4 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |

指数部 $e$　　仮数部 $m$　　丸め誤差

$$\text{丸め誤差} = 1234.56 - 1234.5678 = -0.0078$$

$$\text{相対誤差} = \frac{-0.0078}{1234.5678} = -0.00000631 \quad (\text{精度 6 桁})$$

- 10 進小数 $0.1$ を IEEE 単精度浮動小数点数で表すと

$$0.1 \times 2 = 0.2 + 0$$

$$0.2 \times 2 = 0.4 + 0$$

$$0.4 \times 2 = 0.8 + 0$$

$$0.8 \times 2 = 0.6 + 1$$

$$0.6 \times 2 = 0.2 + 1$$

$$0.2 \times 2 = 0.4 + 0 \quad (\text{以下、同じ計算の繰り返し})$$

$$\vdots$$

$$\therefore 0.1 = (0.0\underbrace{0011}_{\text{循環の単位}}0011\cdots)_2 \simeq (1.\underbrace{10011\cdots}_{\text{23 桁}})_2 \times 2^4$$

$$\text{丸め誤差} = (-0.\underbrace{0\cdots0}_{\text{23 桁}}\underbrace{1100\cdots}_{\text{丸め誤差}})_2 \times 2^4$$

# 情報落ち

- 値に大きな差がある数の加減算に起因
- 10 進 8 桁の浮動小数点数 (仮数部が 6 桁) で

$$123.456 + 0.0001$$

を計算

$$\begin{array}{cccccccccccccc}
 & 1 & 2 & 3 & . & 4 & 5 & 6 & & & & & & \\
+ & & & 0 & . & 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\
\hline
 & 1 & 2 & 3 & . & 4 & 5 & 6 & \cancel{1} & \cancel{0} & \cancel{0} & \cancel{0} & \cancel{0} & \cancel{0}
\end{array}$$

$\rightarrow$ 0.0001 の加算が無視される (情報落ち)

加減算を順序を工夫 (e.g. 小さい順) して情報落ちを回避

# 計算機イプシロン

- 浮動小数点数の有効桁数は有限なため、表せる数は不連続
- 1 と「1 の次の数」の差を計算機イプシロン ($\varepsilon$) と呼ぶ
  - $\varepsilon$ より小さな数を 1 に足し引きしても無視 (情報落ち)
  - IEEE 単精度浮動小数点数の場合

$$\begin{aligned}\varepsilon &= (1.\underbrace{0\cdots0}_{22\text{ 桁}}1)_2 - (1.\underbrace{0\cdots0}_{22\text{ 桁}}0)_2 \\ &= (0.\underbrace{0\cdots0}_{22\text{ 桁}}1)_2 \\ &= 2^{-23} \simeq 1.19 \times 10^{-7}\end{aligned}$$

  - IEEE 倍精度浮動小数点数の場合

$$\varepsilon = 2^{-52} \simeq 2.22 \times 10^{-16}$$

# C 言語での計算機イプシロン

- flt_dbl_epsilon.c

```
#include <stdio.h>
#include <float.h>
// FLT_EPSILON, DBL_EPSILON

int main(){
  printf("%.8e\n", FLT_EPSILON);
  printf("%.8e\n", DBL_EPSILON);
}
```

- 実行結果

```
$ gcc flt\_dbl\_epsilon.c
$ ./a.out
1.19209290e-07
2.22044605e-16
```

  - 単精度のとき

$$\varepsilon = 2^{-23}$$

  - 倍精度のとき

$$\varepsilon = 2^{-52}$$

# 計算機イプシロンの確認

- epsilon.c

```
#include <stdio.h>

int main(){
  float eps = 1.0;
  for(int i=0; i<30; ++i){
    printf("%.6e %.12f ",
           eps, 1 + eps);
    printf(1+eps==1?
           "true\n":
           "false\n");
    eps /= 2.0;
  }
}
```

- 実行結果

```
$ gcc epsilon.c
$ ./a.out
1.000000e+00 2.000000000000 false
5.000000e-01 1.500000000000 false
.
.
.
2.384186e-07 1.000000238419 false
1.192093e-07 1.000000119209 false
5.960464e-08 1.000000000000 true
2.980232e-08 1.000000000000 true
```

  - eps = 1.0 を次々に 2 で割る
  - eps $< 2^{-23}$ のとき情報落ち

# 桁落ち

- 値の近い数の引き算に起因
- 10 進 8 桁の浮動小数点数 (仮数部が 6 桁) で $123.456 - 123.444$ を計算すると

$$
\begin{array}{rcccccccccccc}
 & & & & & & \overbrace{\phantom{1\ 2\ 3\ .\ 4\ 5\ 6}}^{\text{有効数字 6 桁}} & & & & & & \\
 & 1 & 2 & 3 & . & 4 & 5 & 6 & * & * & * & * & \cdots \\
- & 1 & 2 & 3 & . & 4 & 4 & 4 & * & * & * & * & \cdots \\
\hline
 & & & 0 & . & 0 & 1 & 2 & * & * & * & * & \cdots \\
 & & & \underbrace{\phantom{0\ .\ 0}}_{\text{4 桁が相殺}} & & & & & \underbrace{\phantom{* * * * \cdots}}_{\text{信用できない数字}} & & & &
\end{array}
$$

$$0.012 * * * * = 0.\underbrace{12 * * * *}_{\text{仮数部}} \times 10^{49-50}$$

上位の有効数字が相殺し、有効桁が 6 桁から 2 桁に減少

# 桁落ちの回避

## 方針

値の近い数の引き算を足し算に変形

- 例えば、引き算

$$\sqrt{x+1}-1$$

は $|x| \ll 1$ のとき $\sqrt{x+1} \simeq 1$ なので桁落ちが発生

- そこで、分母・分子に $\sqrt{x+1}+1$ を掛けて

$$\sqrt{x+1}-1 = (\sqrt{x+1}-1)\cdot\frac{\sqrt{x+1}+1}{\sqrt{x+1}+1} = \frac{x}{\sqrt{x+1}+1}$$

と変形すると引き算の代わりに足し算が出現 → 桁落ちしない

# 桁落ちの有無の比較

- 小さい値 $x = 0.00123456$ に対して有効数字 6 桁で計算
  - そのまま計算

$$\sqrt{x+1} - 1 = \underbrace{1.00062 - 1.00000}_{\text{有効数字 6 桁}} = \underbrace{0.00062}_{\text{有効数字 2 桁}}$$

有効数字が 6 桁から 2 桁に減少

  - 引き算を足し算に変換して計算

$$\frac{x}{\sqrt{x+1}+1} = \frac{0.00123456}{1.00062 + 1.00000} = \underbrace{\frac{0.00123456}{2.00062}}_{\text{有効数字 6 桁}} = 0.000\,\underbrace{617089}_{\text{有効数字 6 桁}}$$

有効数字は 6 桁のまま (掛け算・割り算では有効桁数は不変)

# 2 次方程式の解の公式における桁落ち

- 2 次方程式 $ax^2 + bx + c = 0$ の解

$$x = \frac{-b \pm \sqrt{D}}{2a} \qquad (D = b^2 - 4ac > 0)$$

には 2 箇所の引き算が存在

1. $b^2 - 4ac$
2. $-b \pm \sqrt{D}$

- $b^2 \simeq 4ac$、つまり $D \simeq 0$ のとき $b^2 - 4ac$ の引き算で桁落ち

$\rightarrow$ 1. の桁落ちを回避不可能

- $b^2 \gg 4ac$、つまり $D \simeq |b|$ のとき
  - $b > 0$ ならば

$$\frac{-b + \sqrt{D}}{2a} = \frac{-b + \sqrt{D}}{2a} \cdot \frac{b + \sqrt{D}}{b + \sqrt{D}} = \frac{-2c}{b + \sqrt{D}}$$

  - $b < 0$ ならば

$$\frac{-b - \sqrt{D}}{2a} = \frac{-b - \sqrt{D}}{2a} \cdot \frac{b - \sqrt{D}}{b - \sqrt{D}} = \frac{-2c}{b - \sqrt{D}}$$

$\rightarrow$ 足し算への変形で 2. の桁落ちを回避可能

# 統計学の分散における桁落ち

- 標本 $x_1, x_2, \cdots, x_n$ が得られたとき、標本分散は

$$s^2 = \boxed{\frac{1}{n}\sum_{i=1}^{n}(x_i - \bar{x})^2} \qquad \left(\text{平均} \quad \bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i\right) \tag{1}$$

$$= \frac{1}{n}\sum_{i=1}^{n}({x_i}^2 - 2\bar{x}x_i + \bar{x}^2)$$

$$= \frac{1}{n}\sum_{i=1}^{n}{x_i}^2 - 2\bar{x}\underbrace{\frac{1}{n}\sum_{i=1}^{n}x_i}_{\bar{x}} + \bar{x}^2\underbrace{\frac{1}{n}\sum_{i=1}^{n}1}_{1}$$

$$= \boxed{\frac{1}{n}\sum_{i=1}^{n}{x_i}^2 - \bar{x}^2} \tag{2}$$

- (1),(2) いずれの表式で分散を計算すればよいか?

# 分散の計算例: 1円玉の重さ

- 標本値は参考図書 p.17 参照

▷ variance.c

```
#include <stdio.h>
#include <math.h>   // pow

int main(){
  float data[] = {0.9997, 1.0007, 0.9995, ...};
  int num = sizeof(d) / sizeof(data[0]);

  float mean = 0.0;   // 平均
  for(int i=0; i!=num; ++i)
    mean += data[i];
  mean /= num;

  float var1 = 0.0;   // 分散 (1)
  for(int i=0; i!=num; ++i)
    var1 += pow(data[i] - mean, 2.0);
  var1 /= num;
}
```

```
  float var2 = 0.0;   // 分散 (2)
  for(int i=0; i!=num; ++i)
    var2 += pow(data[i], 2.0);
  var2 /= num;
  var2 -= pow(mean, 2.0);

  printf("%e\n%e\n", var1, var2);
}
```

▷ 出力

```
$ gcc variance.c -lm
$ ./a.out
3.083146e-07   // (1) の分散
1.681441e-07   // (2) の分散
```

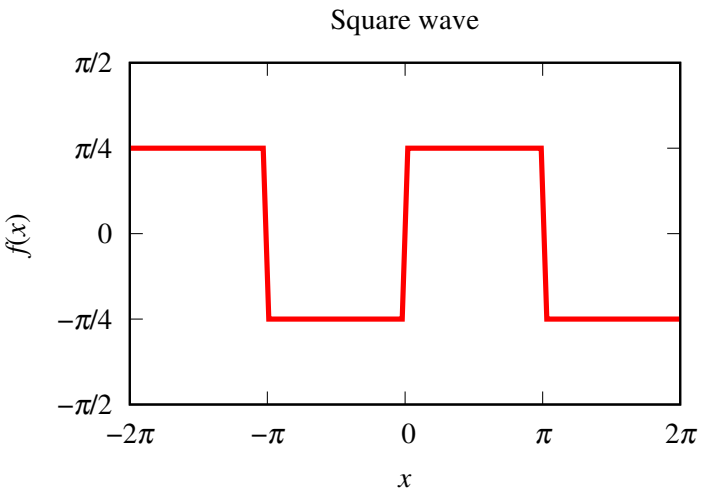
(2) の方が桁落ちが大きい

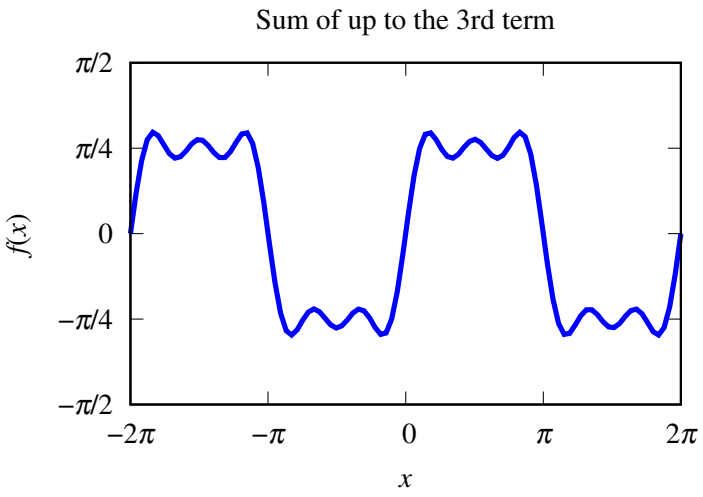
# 打ち切り誤差

- 無限回の演算を有限回で打ち切ることに起因
- 例: 方形波の Fourier 級数展開

$$f(x) = \begin{cases} \frac{\pi}{4} & 2m\pi < x < (2m+1)\pi \\ -\frac{\pi}{4} & (2m-1)\pi < x < 2m\pi \end{cases} \qquad (m = 0, \pm 1, \pm 2, \cdots)$$

$$= \sin x + \frac{1}{3}\sin 3x + \frac{1}{5}\sin 5x + \cdots = \sum_{n=0}^{\infty} \frac{\sin(2n+1)x}{2n+1}$$



- $f(x)$ に $x = \frac{\pi}{2}$ を代入すると

$$\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \cdots = \sum_{n=0}^{\infty} \frac{(-1)^n}{(2n+1)} \qquad \text{(Leibniz の公式)}$$

- 円周率を有限和で近似すると

$$\begin{aligned}
\pi_3 &\simeq 4\left(1 - \frac{1}{3} + \frac{1}{5}\right) &&= 3.46666 > \pi, \\
\pi_4 &\simeq 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7}\right) &&= 2.89523 < \pi, \\
\pi_5 &\simeq 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9}\right) &&= 3.33968 > \pi, \\
\pi_6 &\simeq 4\left(1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11}\right) &&= 2.97604 < \pi, \\
&\vdots
\end{aligned}$$

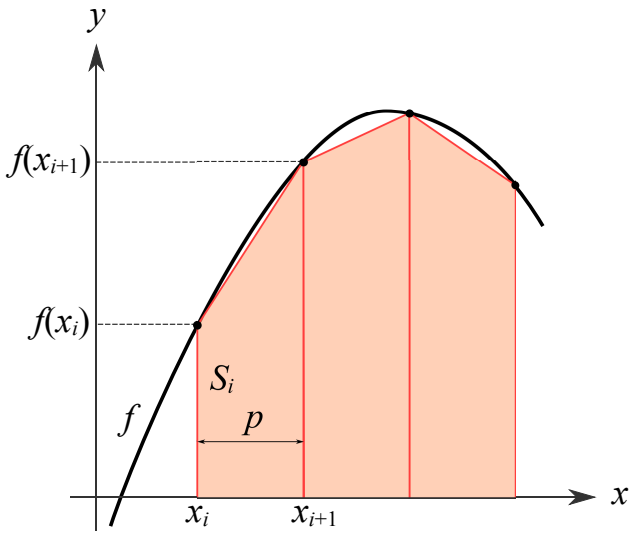
情報落ちを避けるために、高次 (右側) の項から逆順に和を取る

# 離散化誤差

- 連続量を離散値で近似することに起因
- 例: 定積分の台形公式

$$\int_a^b f(x)\mathrm{d}x = \frac{b-a}{N}\left[\frac{f(x_0)}{2} + \sum_{n=1}^{N-1} f(x_n) + \frac{f(x_N)}{2}\right] \quad \left(\begin{array}{l} x_0 = a, \\ x_N = b \end{array}\right)$$

- $N$ が大きくすると
  - 離散化誤差は減少
  - 情報落ち、桁落ちの可能性が増加




出典: Wikipedia

# まとめ

- 精度の見積もり

$$\text{精度の桁数} = -\log_{10} |\text{相対誤差}|$$

- 誤差の種類

| 誤差 | 原因 |
|---|---|
| 丸め誤差 | 有効桁数が有限 |
| 情報落ち | 値に大きな差がある 2 数の加減算 |
| 桁落ち | 値が近い 2 数の減算 |
| 打切り誤差 | 無限回の演算を有限回で打ち切り |
| 離散化誤差 | 連続量を離散値で近似 |